

MODULE 11 · DOMAIN 2 - DATA STORE MANAGEMENT

Choosing a data store

DynamoDB, Aurora, DocumentDB, Keyspaces, Neptune, OpenSearch, MemoryDB, and the vector indexes new to this exam.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 11 of 27

Module one gave you the rule. **Access pattern decides the store.** This module makes it precise.

The stores, in one line each

SERVICE	BUILT FOR
Amazon DynamoDB	Key-value and document. Single-digit millisecond at any scale
Amazon RDS	Managed relational. Six engines, familiar operational model
Amazon Aurora	MySQL and PostgreSQL compatible, rearchitected for the cloud
Amazon DocumentDB	MongoDB-compatible document store
Amazon Keyspaces	Apache Cassandra-compatible, wide column
Amazon Neptune	Graph. Relationships are the query, not the join
Amazon OpenSearch Service	Search and log analytics. Free text and aggregations
Amazon MemoryDB	In-memory, durable. Microsecond reads, Redis-compatible

ON THE EXAM

Four of these exist because a team already uses MongoDB, Cassandra, Redis or PostgreSQL. Compatibility is the answer.

Index design in OpenSearch

- ◆ **Shards divide an index** across nodes, exactly as slices divide a Redshift table.
- ◆ **Too many small shards** waste memory and slow the cluster.
- ◆ **Shard count is fixed at index creation.** Changing it means reindexing.
- ◆ **Index per time period** for logs, so old indices can be deleted whole.
- ◆ **Index State Management** moves indices to cheaper storage and deletes them on a schedule.

ON THE EXAM

Rolling daily or monthly indices is the log pattern. Deleting an index is instant; deleting documents is not.

The four questions that pick a store

- ◆ **How is a record retrieved?** By key, by query, by search, or by relationship.
- ◆ **How fast must it answer?** Microseconds, milliseconds, or seconds.
- ◆ **Is there existing technology?** MongoDB, Cassandra, Redis and PostgreSQL each map to a service.
- ◆ **Read one thing or aggregate many?** Operational versus analytical.
- ◆ The fourth is the one that separates every store here from Amazon Redshift.

ON THE EXAM

Everything in this module is operational. Redshift and Athena are analytical. Mixing those up is the biggest error available.

DynamoDB is fast because it makes you **decide the queries first**. That is the whole trade.

Keys, and what they determine

- ◆ **Partition key** decides which physical partition an item lives on.
- ◆ **Sort key** orders items within a partition, and enables range queries.
- ◆ **Together they are the primary key**, and together they must be unique.
- ◆ **Query** reads items by partition key, optionally filtered by sort key. Fast.
- ◆ **Scan** reads the whole table and filters afterwards. Expensive at any size.

ON THE EXAM

A Scan in an option is nearly always the wrong answer. If a query needs a Scan, the table is designed wrong.

Consistency and durability options

- ◆ **Eventually consistent reads** are the default, and cost half as much.
- ◆ **Strongly consistent reads** always return the latest write, at double the cost.
- ◆ **Global secondary indexes are eventually consistent only.** There is no strong option.
- ◆ **Transactions** group several writes so all succeed or none do.
- ◆ **Point-in-time recovery** restores to any second in the last thirty five days.

ON THE EXAM

The GSI consistency limit is the one that appears in questions. If a read must be strongly consistent, it cannot come from a GSI.

Global or local secondary index?

GLOBAL SECONDARY INDEX

A different partition key

- Any attribute as partition and sort key
- Its own capacity, eventually consistent
- Can be added after the table exists
- Up to twenty per table

LOCAL SECONDARY INDEX

Same partition key, different sort

- Alternative sort key only
- Shares the table's capacity, strongly consistent option
- Must be created with the table
- Up to five per table

THE TIEBREAKER

LSIs cannot be added later. That single constraint decides most questions offering both.

Capacity, and the hot partition problem

- ◆ **On-demand** scales automatically and bills per request. For unpredictable load.
- ◆ **Provisioned** bills for reserved capacity. Cheaper for steady, known throughput.
- ◆ **A hot partition key** concentrates traffic and throttles, while the table looks under-used.
- ◆ **Adaptive capacity** shifts throughput toward busy partitions automatically.
- ◆ **The real fix is a better key**, with higher cardinality or a suffix.

ON THE EXAM

This is the Kinesis hot shard problem from module three, in a different service. The diagnosis and fix are identical.

A gaming company stores player profiles in Amazon DynamoDB with player ID as the partition key. A new feature must show a leaderboard of the top one hundred players per region, refreshed constantly and read by every player who opens the app.

The current implementation scans the whole table and sorts the results in the application. The table holds forty million players and read costs have become the largest line on the bill.

What should the data engineer implement?

- ☐ A Increase the table's provisioned read capacity so the scan completes faster
- ☐ B Add a global secondary index partitioned by region and sorted by score
- ☐ C Add a local secondary index sorted by score
- ☐ D Export the table to Amazon S3 nightly and query the leaderboard with Amazon Athena

◆ PAUSE AND DECIDE

What should the data engineer implement?

A More read capacity

Pays more to keep scanning forty million items. It makes the largest line on the bill larger.

C LSI sorted by score

An LSI keeps player ID as the partition key, so it cannot group by region. It also cannot be added to an existing table.

B GSI on region, sorted by score

Turns the scan into a query on one partition, returning the top hundred directly. The access pattern becomes cheap.

D Nightly export to Athena

The leaderboard must refresh constantly. A nightly export is up to twenty four hours stale.

THE SIGNAL

A Scan in the current design plus a new access pattern is the standard shape of a secondary index question.

Amazon RDS or Amazon Aurora?

AMAZON RDS

Managed engines

- MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2
- Familiar, and closest to on-premises behaviour
- Storage attached to the instance
- Choose it when the engine is not MySQL or PostgreSQL

AMAZON AURORA

Rearchitected

- MySQL and PostgreSQL compatible only
- Distributed storage across three AZs
- Up to fifteen low-latency read replicas
- Aurora serverless scales capacity automatically

THE TIEBREAKER

Aurora is the answer when the engine is MySQL or PostgreSQL and performance or availability matter. RDS covers the rest.

Aurora features that appear in questions

- ◆ **Aurora serverless** scales capacity automatically, and can scale to zero.
- ◆ **Aurora Global Database** replicates to other Regions with typical lag under a second.
- ◆ **Zero-ETL to Redshift** makes operational data analytical without a pipeline.
- ◆ **pgvector on Aurora PostgreSQL** stores embeddings beside relational data.
- ◆ **Renamed.** Aurora Serverless v2 became simply Aurora serverless in April 2026.

ON THE EXAM

For a data engineer, the two that matter most are zero-ETL to Redshift and pgvector. Both appear later in this module.

The compatibility stores

SERVICE	COMPATIBLE WITH, AND USED FOR
Amazon DocumentDB	MongoDB. JSON documents, flexible schema, nested structures
Amazon Keyspaces	Apache Cassandra. Wide column, very high write throughput
Amazon MemoryDB	Redis. In-memory speed with durability, unlike a plain cache
Amazon Neptune	Graph. Gremlin, openCypher and SPARQL

ON THE EXAM

MemoryDB is durable. That is what distinguishes it from a cache, and it is why it is the one in scope for this exam.

Amazon OpenSearch Service

- ◆ **Free text search** with relevance ranking, not exact matching.
- ◆ **Log analytics.** The standard destination for application and infrastructure logs.
- ◆ **Aggregations over indexed fields**, which is why dashboards sit on top of it.
- ◆ **OpenSearch Serverless** removes the cluster, billed in OCUs.
- ◆ **Vector search** is built in, which makes it the default production RAG store.

ON THE EXAM

Search means OpenSearch. Not Athena, not DynamoDB. Relevance-ranked text retrieval is what it exists for.

A retailer runs a product catalogue on a self-managed MongoDB cluster on Amazon EC2. Around sixty microservices query it using MongoDB drivers. The operations team spends significant time patching and managing replica sets, and wants to reduce that work.

A proposal on the table is to migrate the catalogue to Amazon DynamoDB, on the grounds that it is fully managed and scales better.

How should the data engineer respond?

- ☐ A Agree. DynamoDB is fully managed and eliminates the operational work
- ☐ B Recommend Amazon DocumentDB, which is MongoDB compatible and leaves the sixty services unchanged
- ☐ C Recommend Amazon Aurora PostgreSQL, using JSONB columns for the documents
- ☐ D Recommend Amazon Keyspaces, which handles document workloads at scale

◆ PAUSE AND DECIDE

How should the data engineer respond?

- A

 Agree, migrate to DynamoDB
Removes the operational work by rewriting sixty services onto a different API and a different data model.
- B

 DocumentDB, MongoDB compatible
Same drivers, same queries, no application changes, and AWS operates it. Exactly what the service exists for.
- C

 Aurora PostgreSQL with JSONB
Technically capable of storing documents, and every one of the sixty services would need rewriting.
- D

 Keyspaces
Cassandra compatible, not MongoDB compatible. Wide column is a different model from documents.

THE SIGNAL

Sixty services using MongoDB drivers is a compatibility requirement, exactly like existing Kafka meaning MSK.

An analytical schema and an operational schema are shaped by **opposite forces**.

Normalised or denormalised?

NORMALISED

Operational

- Each fact stored once, referenced by key
- Updates touch one row
- Reads need joins
- Third normal form, roughly

DENORMALISED

Analytical

- Values repeated to avoid joins
- Updates touch many rows
- Reads are fast and simple
- Star schema, or a single wide table

THE TIEBREAKER

A star schema is denormalisation with discipline. Facts in the centre, dimensions around it.

Designing a DynamoDB table

- ◆ **List the access patterns first**, before any table exists.
- ◆ **Choose a partition key** that spreads traffic and matches the common query.
- ◆ **Composite sort keys** encode hierarchy, such as an order date and status.
- ◆ **Single-table design** stores several entity types in one table, keyed to be queried together.
- ◆ **Do not normalise**. Joins do not exist, so a join becomes several round trips.

ON THE EXAM

The relational instinct to split entities into separate tables is the most common DynamoDB design mistake.

A logistics company needs to answer questions like which drivers have delivered to which depots, and how two shipments are connected through shared handling facilities. The relationships are several layers deep and the depth varies by question.

The data currently sits in Amazon RDS for PostgreSQL, and these queries require recursive joins that take minutes to return.

Which store best fits this access pattern?

- ☐ A Amazon Neptune, querying the relationships as a graph
- ☐ B Amazon DynamoDB, with a global secondary index per relationship type
- ☐ C Amazon Redshift, loading the tables and using window functions
- ☐ D Amazon OpenSearch Service, indexing the relationships as documents

◆ PAUSE AND DECIDE

Which store best fits this access pattern?

- A

 Amazon Neptune
Traversing relationships of varying depth is what a graph database does natively. It is one query, not a recursive join.
- B

 DynamoDB with indexes per relationship
Each hop becomes another round trip. Variable depth means an unknown number of them.
- C

 Redshift with window functions
An analytical warehouse. Window functions rank and aggregate; they do not traverse arbitrary-depth relationships.
- D

 OpenSearch
Finds documents matching terms. It has no notion of following an edge from one document to another.

THE SIGNAL

"Several layers deep" and "depth varies" is the graph signal. No other store answers that in one query.

Designing for Lake Formation

- ◆ **Tables in the Glue Data Catalog** are the unit Lake Formation grants on.
- ◆ **Group columns by sensitivity**, so a column-level grant is meaningful.
- ◆ **Separate tables by audience** when the whole dataset differs, rather than filtering every query.
- ◆ **Partition on a governed attribute** when row-level rules follow it, such as region.
- ◆ A wide table mixing public and restricted columns forces column-level rules on every consumer.

ON THE EXAM

Schema design and access design are the same decision in a lake. Module eighteen covers the permissions themselves.

Vectors are how a machine stores **meaning**, and the exam now asks about them.

What an embedding is, and why it matters

- ◆ **A model converts content into a vector**, often hundreds or thousands of numbers.
- ◆ **Similar meaning means nearby vectors**, even with no words in common.
- ◆ **Similarity is measured** by cosine distance, Euclidean distance or dot product.
- ◆ **Retrieval augmented generation** finds relevant chunks and feeds them to a model.
- ◆ **Keyword search misses synonyms**. Vector search finds them, and can miss exact terms.
- ◆ **Hybrid search** combines both, which is why OpenSearch is the common production choice.

ON THE EXAM

A query for "laptop won't charge" should find "notebook battery not working". No keyword overlaps.

The two index types the exam names

INDEX	CHARACTER
HNSW	A navigable graph of vectors. Fast queries, high recall, more memory
IVF	Vectors clustered into lists. Smaller and cheaper, slightly lower recall
Flat	No index. Exact search, and it compares against every vector

ON THE EXAM

HNSW for query speed and accuracy. IVF when memory or index build time matters more.

Where vectors can live on AWS

SERVICE	WHEN IT FITS
Amazon OpenSearch Service	The production default. Hybrid search, filtering, HNSW at scale
Aurora PostgreSQL with pgvector	Embeddings beside relational data. HNSW and IVFFlat
Amazon DocumentDB	HNSW and IVFFlat, alongside documents
Amazon MemoryDB	Smallest corpus, lowest latency. Single-digit milliseconds
Amazon Neptune Analytics	Graph relationships plus vector similarity together
Amazon S3 Vectors	Cost-optimised vector storage in object storage. Higher latency

ON THE EXAM

Amazon S3 Vectors is the newest option, cutting vector storage cost substantially at the price of latency.

Bedrock Knowledge Bases

- ◆ **Managed retrieval augmented generation.** Point it at data, ask questions in natural language.
- ◆ **It handles chunking, embedding and indexing,** which is the pipeline you would otherwise build.
- ◆ **Supported vector stores** include OpenSearch Serverless, Aurora PostgreSQL and S3 Vectors.
- ◆ **Structured data retrieval too,** not only unstructured documents.
- ◆ **Less control** over chunking strategy than building it yourself.

ON THE EXAM

The skill named in the exam guide is vectorisation concepts and knowledge bases. The overhead argument is the exam point.

An insurance company runs its policy administration system on Amazon Aurora PostgreSQL. It wants to add semantic search over policy documents, so that an adjuster searching for cover for water damage from a burst pipe finds relevant clauses even when the wording differs.

There are around two million document chunks. Searches must also filter by policy type and effective date, which are columns already in the Aurora database. The team is small and wants to avoid running another system.

What should the data engineer recommend?

- ☐ A Store the embeddings in Aurora PostgreSQL using pgvector with an HNSW index
- ☐ B Deploy an Amazon OpenSearch Service domain and index the embeddings there
- ☐ C Store the embeddings in Amazon MemoryDB for the lowest possible latency
- ☐ D Use Amazon Athena to search the document text stored in Amazon S3

◆ PAUSE AND DECIDE

What should the data engineer recommend?

A pgvector with HNSW in Aurora

Embeddings sit beside the columns being filtered on, so it is one query in one system the team already runs.

C MemoryDB

Lowest latency, at in-memory prices for two million vectors, and nothing here requires microseconds.

B An OpenSearch domain

The strongest choice at large scale. Here it means a second system to run and the filter columns living elsewhere.

D Athena over document text

Keyword matching over files. The whole requirement is finding matches where the wording differs.

THE SIGNAL

The filter columns already live in Aurora. That is what turns two systems into one.

A media company is building a support assistant over eight hundred thousand help articles. It needs semantic retrieval, keyword matching for exact product codes, filtering by product line, and relevance tuning. Query volume is high and latency must stay under two hundred milliseconds.

The company has a platform team with capacity to operate infrastructure.

Which vector store fits best?

- A Amazon S3 Vectors, for the lowest storage cost at this corpus size
- B Amazon OpenSearch Service with HNSW indexing and hybrid search
- C Aurora PostgreSQL with pgvector and an IVFFlat index
- D Amazon Neptune Analytics, treating articles as connected nodes

◆ PAUSE AND DECIDE

Which vector store fits best?

A S3 Vectors

Cheapest storage, and its latency is measured well above the stated two hundred millisecond budget.

C pgvector with IVFFlat

Workable, and no relational data is involved here, and IVF trades recall for memory in a system built for accuracy.

B OpenSearch with HNSW and hybrid search

Semantic and keyword together is exactly hybrid search, plus filtering and relevance tuning, at the required latency.

D Neptune Analytics

Graph plus vectors. Nothing in the requirement is about relationships between articles.

THE SIGNAL

Compare with scenario 4. Same technology, different answer, because the requirements point elsewhere.

What each store charges for

SERVICE	BILLING
DynamoDB on-demand	Per read and write request, plus storage
DynamoDB provisioned	Reserved capacity units per hour, used or not
Aurora	Instance hours or ACUs, plus storage and I/O
OpenSearch	Instance hours, or OCU on Serverless with a minimum
MemoryDB	Instance hours. In-memory capacity is the expensive part
Amazon S3 Vectors	Storage and requests, at object storage prices

ON THE EXAM

The same rule as always. Continuous billing punishes idle. Per-request billing punishes constant heavy use.

An application writes session state to Amazon DynamoDB with provisioned capacity of four thousand write capacity units. Traffic is heavily concentrated in a two hour window each evening, and near zero for the rest of the day.

During the peak, writes are occasionally throttled. Outside it, utilisation sits below five percent. The team wants to reduce cost and stop the throttling.

What should the data engineer do?

- ☐ A Increase provisioned capacity to eight thousand write capacity units
- ☐ B Switch the table to on-demand capacity mode
- ☐ C Enable DynamoDB auto scaling with the same minimum and a higher maximum
- ☐ D Add a global secondary index to spread the write load

◆ PAUSE AND DECIDE

What should the data engineer do?

- A

 Double the provisioned capacity
Stops the throttling and doubles the cost of twenty two idle hours a day. It fixes one requirement and worsens the other.
- B

 Switch to on-demand
Pays per request, so the idle hours cost almost nothing and the peak scales without throttling. Both requirements, one change.
- C

 Auto scaling with a higher ceiling
Genuinely reasonable, and it reacts to load rather than anticipating it, so a sharp ramp still throttles briefly.
- D

 Add a GSI
An index adds write load rather than spreading it. Every write to the table also writes to the index.

THE SIGNAL

Utilisation below five percent for twenty two hours is the on-demand signal. So is throttling despite low average use.

Getting operational data into analytics

- ◆ **Zero-ETL to Redshift** from Aurora, Amazon RDS and DynamoDB.
- ◆ **Zero-ETL to OpenSearch** from DynamoDB, for search over operational data.
- ◆ **DynamoDB export to S3** for a point-in-time snapshot, with no capacity consumed.
- ◆ **Export uses no read capacity**, unlike scanning the table.
- ◆ **DMS with CDC** when the pair is not supported by zero-ETL.

ON THE EXAM

Exporting a DynamoDB table to S3 consumes no read capacity. Scanning it to achieve the same thing consumes a great deal.

A company stores order records in Amazon DynamoDB. The analytics team needs the full order history in the data lake daily for reporting, around two hundred million items. The current process runs a parallel Scan overnight.

The Scan consumes most of the table's provisioned read capacity, and the customer-facing application has started seeing throttled reads during the overnight window.

What should the data engineer implement?

- ☐ A Use DynamoDB export to Amazon S3, which consumes no table read capacity
- ☐ B Increase the table's provisioned read capacity for the overnight window
- ☐ C Run the Scan with a lower rate limit so it takes longer but uses less capacity
- ☐ D Enable DynamoDB Streams and rebuild the full history from the change records

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

DynamoDB export to S3
Reads from continuous backups rather than the table, so it consumes no read capacity and cannot throttle the application.
- B

More capacity overnight
Pays more to keep competing with the application. It manages the symptom rather than removing the contention.
- C

Rate-limit the Scan
Reduces contention by making the export take longer, and it still consumes capacity the application wants.
- D

Rebuild from Streams
Streams carry changes, not existing state, and retain only twenty four hours. There is no full history to rebuild from.

THE SIGNAL

Contention between an analytical scan and an application is the export signal. It removes the competition entirely.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Throttling while utilisation looks low	A hot partition key concentrating traffic
Read costs dominated by one query	A Scan where the design needs an index
An analytical scan throttles the application	Use export to S3 instead of scanning
Semantic search misses exact codes	Vector search alone. Hybrid search is the fix
A new access pattern needs a new sort key	LSIs cannot be added later. It has to be a GSI
Deep relationship queries are slow	A relational store doing a graph's job

ON THE EXAM

The first row appeared in module three as a Kinesis hot shard. Same diagnosis, same fix, different service.

A company is choosing a store for four workloads. A shopping cart read by customer ID with millisecond latency. A three year sales history queried by analysts with varied ad hoc SQL. A product search where shoppers type free text. A fraud investigation tool tracing connections between accounts to arbitrary depth.

An architect proposes putting all four in Amazon Aurora PostgreSQL for consistency and to reduce the number of systems.

How should the data engineer respond?

- ☐ A Agree. One system is simpler to operate and PostgreSQL can serve all four
- ☐ B Recommend DynamoDB, Redshift, OpenSearch and Neptune respectively, matching each workload to its access pattern
- ☐ C Recommend Amazon Redshift for all four, since it handles both operational and analytical queries
- ☐ D Recommend Aurora for the first two and OpenSearch for the other two

◆ PAUSE AND DECIDE

How should the data engineer respond?

- A

 Everything in Aurora
PostgreSQL can technically do all four. It is good at one, adequate at two, and poor at deep relationship traversal.
- B

 Four stores, one per access pattern
Key lookup, analytical aggregation, relevance-ranked search and graph traversal are four different problems with four purpose-built answers.
- C

 Everything in Redshift
An analytical warehouse serving a millisecond shopping cart. Module one's second scenario, repeated.
- D

 Aurora and OpenSearch
Better than A, and it still puts the fraud graph in a search engine that cannot traverse edges.

THE SIGNAL

Compare with module five's scenario 8. Same instinct to standardise, same failure, different layer of the stack.

Reading a data store question quickly

- ◆ **Operational or analytical?** That eliminates half the options immediately.
- ◆ **Is an existing technology named?** MongoDB, Cassandra, Redis, PostgreSQL each map to one service.
- ◆ **How is the record found?** Key, query, search, or relationship.
- ◆ **Is a latency number given?** Microseconds, milliseconds and seconds have different answers.
- ◆ **For vectors, what else is being filtered?** Relational filters point at pgvector.

ON THE EXAM

Next: cataloguing, schema evolution and lineage. The Glue Data Catalog, crawlers, and SageMaker Catalog.